MUS v2 Concept: “Agent Tasking + Internal Ops”

MUS v2 Concept: “Agent Tasking + Internal Ops”

1) New mental model (simple)

You manage 5 things:

1. **Agents** (who can run tasks, what tools/capabilities they have, budgets)
2. **Tasks** (what to check: bottlenecks, quality depth, drift, regression, etc.)
3. **Runs** (executions: manual or scheduled)
4. **Findings / Insights** (what’s wrong, why, impact)
5. **Recommendations** (proposed changes) → optionally converted into ChangeSets → approvals

Detectors become “checks” inside tasks, but the UX and system center on **Tasks**.

What we change in your current architecture

You already have:

- UI: `App/src/pages/maintenance.tsx` with tabs
- Modern API: `server/mus-routes.ts`
- Compatibility API: `server/routes.ts`
- Core: `Axion/src/core/mus/engine.ts`, `store.ts`, `ledger.ts`

- MCP lifecycle modules: `Axion/src/core/mcp/*` with PLANNED→APPLYING→VERIFYING→COMPLETE

That's actually perfect—**MCP becomes the execution/orchestration layer**, while MUS becomes the governance + task catalog layer.

Key correction:

- **MUS defines**: agents, tasks, schedules, budgets, outputs, approvals.
- **MCP executes**: task plans (and can still use PLANNED→… lifecycle), but the UI shouldn't expose "apply/verify/complete" as the main workflow unless you're actually executing a ChangeSet.

MUS v2 additions (minimal but makes it user-friendly)

MUS-NEW-01: Agent Registry (first-class)

Add a registry + API for "available agents" and "capabilities":

Agent

- `agent_id`, `name`
- `capabilities[]` (e.g., `perf_probe`, `quality_audit`, `template_lint`, `dependency_graph`)
- `allowed_scopes`
- `budgets` (token/time)
- `status` (enabled/disabled)
- `run_policy` (manual_only / scheduled_allowed)

MUS-NEW-02: Task Catalog (first-class)

A **Task** is what you actually want to run.

TaskDefinition

- `task_id`, `name`, `intent`
- `default_agent` (or selection rules)
- `inputs_schema`
- `runbook` (steps the agent follows)
- `outputs` (insights, findings, recommendations, metrics)
- `budgets/caps`
- `schedule_allowed` flag

Example tasks (matches your intent)

- `TASK-PERF-01` Pipeline Bottleneck Finder
- `TASK-QUAL-01` “Increase Detail Depth” Audit
- `TASK-QUAL-02` Template Quality + Determinism Audit
- `TASK-SYS-01` Internal Integrity / Drift Scan
- `TASK-OPS-01` Reliability Regression Watch (scheduled)
- `TASK-COST-01` Token/Compute Waste Hotspots

MUS-NEW-03: New output types (beyond Findings)

Right now MUS outputs are findings/proposals/proofs. Add:

- **Insights**: narrative + evidence + ranked hypotheses
- **Bottlenecks**: hotspots with “where time/token goes” + stage-level breakdown
- **Quality Opportunities**: “where depth/detail could be increased” with specific targets
- **Recommendations**: bounded proposals (still can convert into ChangeSets)

This is what makes it feel like “internal monitoring/troubleshooting.”

MUS-NEW-04: Runs become “Task Runs”

Extend `MaintenanceRun` to contain:

- `task_ids[]` (instead of only detector packs)
- `assigned_agents[]`
- `telemetry_summary` (time/token/cost)
- `outputs`: findings, insights, bottlenecks, recommendations, proofs

MUS-NEW-05: UI becomes “Ops Console”

Keep your 7 tabs, but **rename/reframe** and add one:

- Overview ✅
- **Tasks** (new) ← browse tasks, run now, assign agent, set scope/budget
- **Runs** (your “Run” tab becomes run history + run launcher)
- Findings ✅
- **Insights** (new) ← bottlenecks/quality depth/reliability summaries
- Proposals ✅ (recommendations live here too)
- Approvals ✅

- Schedules ✅
- Registries ✅

If you don't want more tabs: merge **Insights** into Overview and keep "Tasks" as the new main tab.

API updates (small, consistent with your existing routes)

Add to `server/mus-routes.ts` (modern MUS routes):

Agents / Tasks

- `GET /api/mus/agents`
- `GET /api/mus/tasks`
- `POST /api/mus/task-runs` (create run: {task_ids, agent_ids?, scope, budgets, trigger})
- `POST /api/mus/task-runs/:runId/start`
- `GET /api/mus/task-runs`
- `GET /api/mus/task-runs/:runId/outputs` (findings + insights + bottlenecks + recommendations)

Insights index

- `GET /api/mus/insights` (latest summaries per task/run)
- `GET /api/mus/bottlenecks` (latest bottleneck reports)

Compatibility routes can remain read-only.

Core engine update (where the “agents” plug in)

In `Axion/src/core/mus/engine.ts`:

- add a **TaskRunner** layer that:
    1. loads task definition
    2. selects/assigns agent
    3. executes runbook (can call existing detector packs, MCP managers, profilers, analyzers)
    4. emits outputs (insights/bottlenecks/recommendations)
    5. persists via `store.ts`
    6. logs via `ledger.ts`

In `Axion/src/core/mcp/modeRunner.ts`:

- treat MUS tasks as “plans” MCP can execute with the existing lifecycle.
- MCP managers (dependencyManager, testMaintainer, ciMaintainer, etc.) become reusable *task executors*.

What you get after this update (what “user friendly” means)

- You click **Tasks** → **Bottleneck Finder** → **Run now**

- It assigns an agent, runs analysis, returns:
    - “top 5 bottlenecks”
    - “where depth can be increased”
    - “recommended patches” (proposal packs)
- You accept recommendations into a ChangeSet
- Approvals gate apply/publish (unchanged)
- You can schedule a “weekly internal monitor” task (proposal-only)

MUS v2 Delta Plan

MUS v2 Delta Plan — Agent Tasking + Internal Ops Console

North Star UX (what the page should feel like)

You can:

- pick a **Task** ("Find bottlenecks", "Increase detail depth", "Audit quality drift")
- assign an **Agent** (or auto-pick by capability)
- run it **now** or on an **allowed schedule**
- get **Insights** (not just findings): bottlenecks, hypotheses, depth opportunities, suggested upgrades
- optionally convert recommendations → ChangeSet → Approvals (unchanged)

MCP lifecycle stays, but becomes an internal execution detail; UI is **Task-first**, not "apply/verify/complete" first.

---

1) Data Model Update (MUS Core)

Add these *new first-class objects* (small, clean):

1.1 AgentDefinition

- `agent_id`, `name`, `capabilities[]`, `status`
- `allowed_scopes`, `budgets` (token/time)
- `schedule_allowed` (bool)

1.2 TaskDefinition

- `task_id`, `name`, `intent` (`monitor|troubleshoot|optimize|audit`)
- `required_capabilities[]`
- `default_agent_id` (optional)
- `inputs_schema` (scope/budgets/targets)
- `outputs` enabled: `insights|bottlenecks|recommendations|findings|proofs`

- `schedule_allowed` (bool)

1.3 TaskRun (extends current MaintenanceRun)

Keep `mode_id` for compatibility, but TaskRun becomes primary:

- `run_id`, `trigger`, `status`
- `task_ids[]`, `assigned_agents[]`
- `scope_selectors`, `budgets`
- `outputs_refs`:
  - `findings.json`
  - `insights.json` ✅ new
  - `bottlenecks.json` ✅ new
  - `recommendations.json` (or reuse proposal_packs.json)
  - `proof_bundle.json`

1.4 Insight / BottleneckReport

- Insight: narrative + evidence pointers + confidence + suggested next actions
- BottleneckReport: top hotspots + stage/time/token breakdown + “why” hypotheses + evidence

Important: proposals remain bounded + consent-gated; we’re just adding a *human-usable layer* above them.

---

2) Registries (minimal additions)

Add two new registries next to your existing ones:

- `REG-AGENTS.json`
- `REG-TASKS.json`

Seed with 5–8 tasks that match your intent:

- `TASK-PERF-01` Pipeline Bottleneck Finder
- `TASK-QUAL-01` Detail Depth Opportunities
- `TASK-QUAL-02` Template Determinism + Quality Audit
- `TASK-OPS-01` Internal Monitor (lightweight scheduled)
- `TASK-COST-01` Token/Compute Waste Hotspots

This is what makes it “user friendly”: you select tasks by name, not modes/units.

---

3) Engine / MCP Integration (keep your structure, change the interface)

3.1 MUS Engine becomes “Task Orchestrator”

In `Axion/src/core/mus/engine.ts` (per your description), add:

- `listAgents()`, `listTasks()`
- `createTaskRun()`, `startTaskRun()`
- `executeTask(task_id, context)` → emits insights/bottlenecks/recommendations

3.2 Reuse MCP managers as task executors

You already have specialized managers: `dependencyManager.ts, migrationManager.ts, refactorManager.ts, testMaintainer.ts, ciMaintainer.ts, axionIntegration.ts` (your list)

Map tasks to executors:

- `TASK-PERF-01` → `axionIntegration + pipeline telemetry + stage timing hooks`
- `TASK-QUAL-01` → template + artifact analyzers (heuristics first)
- `TASK-COST-01` → token/cost signals per stage, cache hit rates, rerun frequency

Key change: MCP lifecycle stays, but it runs **task plans**, not “units”.

---

4) API Changes (small and additive)

You already have:

- Modern: `/api/mus/*`
- Compatibility: `/api/maintenance/*`

Add **only** what's needed for tasking:

4.1 New routes (modern MUS)

- `GET /api/mus/agents`
- `GET /api/mus/tasks`
- `POST /api/mus/task-runs` (create)
- `POST /api/mus/task-runs/:runId/start`
- `GET /api/mus/task-runs`
- `GET /api/mus/task-runs/:runId/outputs` (findings + insights + bottlenecks + recommendations)

Keep existing proposals/changesets/approvals routes. Recommendations can be stored as proposal packs or a sibling object.

4.2 Keep existing mode routes but demote them in UI

Modes still exist for "system maintenance modes", but **Tasks are the primary user entrypoint**.

---

5) Maintenance Page Rewrite (the part that fixes "lost in translation")

Right now your page tabs and behaviors are closer to a maintenance executor and still show Apply/Verify/Complete/Rollback actions

maintenance

, and your Runs tab creates runs with `units` and `intent_type/risk_class/baseline_revision`

maintenance

.

5.1 Replace the primary "Run" UX with "Tasks"

Minimum UI changes (no bloat):

- Rename tab "Runs" → **Task Runs**

- Add a **Tasks** launcher panel at the top of Runs tab (or new tab if you insist)

Task Run Launcher fields (simple)

- Task(s) dropdown
- Agent dropdown (auto-select based on capability)
- Scope selectors
- Budgets/caps
- Run now button

5.2 Add an “Insights” view (can be a panel, not a tab)

Where the value shows up:

- “Top Bottlenecks” panel
- “Depth Opportunities” panel
- “Quality/Detail improvement suggestions”
- “Recommended next task runs”

5.3 Remove Apply/Verify/Complete from the main run list

Those actions are only relevant **after** you convert recommendations → ChangeSet and request approvals.
So:

- Runs show: `queued/running/completed/failed/completed_with_limits`
- Separate section: ChangeSets + Approvals (keep your consent model)

This is the single biggest usability fix.

---

6) What we keep exactly as-is

- Consent gates / approvals concept (Apply vs Publish)
- Scheduled runs toggles (but now schedules schedule **tasks**)
- Audit ledger / proofs
- Existing compatibility endpoints for raw registries/policies

---

7) Implementation Order (fastest to “it works”)

1. **Add registries:** `REG-AGENTS`, `REG-TASKS` + seeds
2. **Add engine primitives:** list agents/tasks, create/start task runs, persist outputs
3. **Add API endpoints:** `/api/mus/agents`, `/api/mus/tasks`, `/api/mus/task-runs*`
4. **Update UI:** replace unit-based run creator + remove apply/verify/complete from run cards
   maintenance

   maintenance
5. **Add one real task implementation first:** `TASK-PERF-01` (bottlenecks) with meaningful output

---

Single prompt you can give your agent (to execute this delta cleanly)

Update MUS to v2: Agent Tasking + Internal Ops.

GOAL
Make Maintenance user-friendly by making "Tasks" the primary unit of work:
- Assign agents to tasks like bottleneck detection and quality depth improvement.
- Runs produce Insights/Bottlenecks/Recommendations, not "apply/verify/complete" workflows.
- Keep consent gates and approvals; never auto-apply/publish.

DO THIS IN MINIMAL CHURN ORDER
1) Add new registries:
  - REG-AGENTS.json
  - REG-TASKS.json
  Seed with 3 agents and 5 tasks (TASK-PERF-01, TASK-QUAL-01, TASK-QUAL-02, TASK-OPS-01, TASK-COST-01).

2) Core types + store:
  - Add AgentDefinition, TaskDefinition, TaskRun, Insight, BottleneckReport models.
  - Extend existing mus store to persist task run outputs:
    insights.json, bottlenecks.json, recommendations.json (or proposal_packs.json).

3) Engine:

- Add listAgents(), listTasks(), createTaskRun(), startTaskRun().
- Implement TASK-PERF-01 first with real output (even heuristic-based):
  stage timing hotspots, rerun hotspots, token/cost hotspots.
- Keep existing MM-01/MM-04 pathways working; map MM-04 to a drift task internally if needed.

4) API (server/mus-routes.ts):
- GET /api/mus/agents
- GET /api/mus/tasks
- POST /api/mus/task-runs
- POST /api/mus/task-runs/:runId/start
- GET /api/mus/task-runs
- GET /api/mus/task-runs/:runId/outputs

5) UI (App/src/pages/maintenance.tsx):
- Remove unit-based run creation and Apply/Verify/Complete/Rollback buttons from the primary run list.
- Add a Task Run launcher: select task(s), agent, scope, budgets, run now.
- Add an Insights section (can be a panel) showing bottlenecks + depth opportunities.
- Keep Proposals/Approvals/Schedules/Registries tabs. Approvals stay required for any apply/publish.

CONSTRAINTS
- No continuous automations.
- Schedules can run tasks, but scheduled runs are proposal-only and cannot publish.
- Automation actors cannot apply/publish.
- Preserve existing UI styling components (GlassPanel/MetricCard/StatusChip).

MUS v2 update

What you have right now (so we update the right thing)

Backend: `/api/mus/*` is “MM run” based

- Create run: only supports `MM-01` and `MM-04` in v1.
  mus-routes
- Start run: calls `executeRun(...)`.
  mus-routes
- Everything else (findings, proposals, blast radius, proof) is keyed off that run model.
  mus-routes

Engine: “detector pack” execution hardcoded to two packs

- `MM-01` runs `DP-REG-01`; `MM-04` runs `DP-DRIFT-01`.
  engine
- Drift detection is currently mostly registry hygiene (slug mismatch, broken references, missing fields).
  engine

UI: the uploaded `maintenance.tsx` is still legacy “unit lifecycle”

- It shows “Units” and lifecycle actions Apply/Verify/Complete/Rollback.
  maintenance

So: to get what you want, we **add agent tasking on top** of existing MUS (don’t break MM runs), then refactor the UI to be **Task-first**.

MUS v2: Minimal change that unlocks “assign agents to tasks”

1) Add two new registries (read-only like your other registries)

- `REG-AGENTS.json`
- `REG-TASKS.json`

The tasks are what you actually click in the UI:

- bottleneck finder
- quality depth increase audit

- template detail/coverage audit
- etc.

2) Add a new run type: TaskRun

Do **not** replace `MusRun` yet; extend it.

Add optional fields:

- `run_kind: "mode" | "task"` (default `"mode"`)
- `task_ids?: string[]`
- `agent_id?: string`
- `outputs?: { insights?: boolean; bottlenecks?: boolean; recommendations?: boolean }`

This keeps the existing run storage intact while enabling tasks.

3) Add new outputs (this is the “user friendly” part)

In addition to findings/proposals, tasks produce:

- `insights.json` (ranked observations + evidence)
- `bottlenecks.json` (hotspots, stage timing/token hotspots)
- `recommendations.json` (can be translated into proposal packs)

No auto-apply/publish. Approvals stay unchanged.

Concrete changes to YOUR files

A) Update `server/mus-routes.ts` to add agent/task endpoints

Right now `mus-routes.ts` loads registries from `Axion/libraries/maintenance/registries/*.json`.

mus-routes

We’ll reuse that with two new endpoints:

Add:

- `GET /api/mus/agents` → reads `REG-AGENTS`
- `GET /api/mus/tasks` → reads `REG-TASKS`
- `POST /api/mus/task-runs` → creates a run with `run_kind:"task"` (instead of mode_id)
- `POST /api/mus/task-runs/:runId/start` → calls a new engine function `executeTaskRun(...)`
- `GET /api/mus/task-runs` → list only task runs
- `GET /api/mus/task-runs/:runId/outputs` → returns findings + insights + bottlenecks + proposals + proof

Why additive? Because your current `/api/mus/runs` is already stable and working.

mus-routes

B) Update `Axion/src/core/mus/engine.ts` to add task execution

Keep `executeRun()` as-is (MM-01/MM-04).

engine

Add a sibling:

- `executeTaskRun(root, store, run)`
  - validates the task exists in `REG-TASKS`
  - selects agent (explicit agent_id or default in task)
  - runs a **task dispatcher**:
    - `TASK-PERF-01` → bottleneck scan
    - `TASK-QUAL-01` → quality/depth opportunities scan
  - saves:
    - findings (optional)
    - insights
    - bottlenecks
    - recommendations (optionally as proposal packs)
    - proof bundle

C) What tasks can do *immediately* (even before deep profiling exists)

You asked for bottlenecks / detail depth improvements. Here's the MVP behavior that's real and useful:

`TASK-PERF-01` Bottleneck Finder (MVP)

- Parse **historical MUS runs** in the store:
  - longest durations (use `started_at/completed_at` timestamps)
  - runs repeatedly hitting `completed_with_limits`
  - detectors producing highest finding counts
- Output `bottlenecks.json`:
  - "top 5 recurring bottlenecks" (e.g., frequent drift, registry issues)
  - "where budgets are being hit" (proposal count caps, max_findings)
- This is implementable right away because the run model already stores counts + status. engine

`TASK-QUAL-01` Increase Detail Depth (MVP)

- Scan templates / knowledge items (if present in registries) for:
  - short descriptions / missing required sections
  - missing `status` / `version` (you already detect this in drift) engine
- Output `insights.json`:
  - "top areas where detail depth is low" (by category)
  - "recommended enhancements" (as recommendation objects)
- Optionally convert some recommendations into bounded proposals (existing proposal pack mechanism).

This gives you immediate internal "monitoring & troubleshooting" without inventing new complexity.

The UI fix (why it feels wrong today)

The uploaded `maintenance.tsx` is still operating the older pipeline model with "units" and lifecycle actions.

maintenance

To match MUS v2:

- Replace "Units + Apply/Verify/Complete" with:
  - **Tasks** picker
  - **Assign Agent**
  - **Run Now**
  - outputs panels: **Insights / Bottlenecks / Recommendations**
- Keep "Approvals" only for ChangeSets/Publish as you already do.

Prompt

You are the Axion Build Agent. Implement MUS v2 ("Agent Tasking + Internal Ops") with minimal churn, using the existing MUS/MM run engine as-is and adding a new TaskRun path on top.

FILES YOU MUST MODIFY
- Backend routes: /mnt/data/mus-routes.ts (mirror changes into repo's server/mus-routes.ts)
- Core engine: /mnt/data/engine.ts (mirror changes into repo's Axion/src/core/mus/engine.ts)

FILES YOU MAY ALSO TOUCH (ONLY IF REQUIRED)
- Axion/src/core/mus/store.ts (add helpers for new outputs if needed)
- Axion/src/core/mus/ledger.ts (log new task-run events)
- Compatibility routes are optional; keep them stable.

NON-NEGOTIABLE BEHAVIOR
1) Proposal-first: tasks may generate findings/insights/bottlenecks/recommendations and optionally proposals, but MUST NOT apply/publish automatically.
2) Consent gates unchanged: Apply and Publish remain explicit approvals (already in system).
3) Scheduling constraint unchanged: no continuous automations; scheduled runs are allowed only if schedule enabled; scheduled runs can NEVER publish.
4) Automation actor can NEVER apply/publish.

GOAL
Make the Maintenance system user-friendly by adding:
- Agent registry
- Task catalog
- Task runs that produce Insights/Bottlenecks/Recommendations (not just findings/proposals)

KEEP EXISTING MUS RUNS WORKING
Do NOT break:
- POST /api/mus/runs (MM-01/MM-04)
- POST /api/mus/runs/:runId/start
- GET /api/mus/runs and GET /api/mus/runs/:runId/findings
- POST /api/mus/changesets, approvals, validate, etc.

PART A — DATA (REGISTRIES)

Add two new registry files under the same MUS_ROOT registry folder used by mus-routes.ts:
- registries/REG-AGENTS.json
- registries/REG-TASKS.json

Use the same registry envelope shape as other REG-*.json (registry_id, registry_version, items[], active_map, invariants, updated_at).
Seed minimal entries:

REG-AGENTS seed (3 agents):
1) AGT-OPS-01
  name: "Ops Analyst"
  capabilities: ["monitor", "bottleneck_scan", "quality_audit"]
  status: "active"
  schedule_allowed: true
  default_budgets: { token_cap: 30000, time_cap_ms: 60000 }

2) AGT-QUAL-01
  name: "Quality Auditor"
  capabilities: ["quality_depth", "template_audit", "kl_audit"]
  status: "active"
  schedule_allowed: true
  default_budgets: { token_cap: 30000, time_cap_ms: 60000 }

3) AGT-SYS-01
  name: "System Inspector"
  capabilities: ["registry_integrity", "drift_scan"]
  status: "active"
  schedule_allowed: true
  default_budgets: { token_cap: 20000, time_cap_ms: 45000 }

REG-TASKS seed (5 tasks):
1) TASK-PERF-01
  name: "Pipeline Bottleneck Finder"
  intent: "troubleshoot"
  required_capabilities: ["bottleneck_scan"]
  default_agent_id: "AGT-OPS-01"
  outputs: { insights: true, bottlenecks: true, recommendations: true, findings: true, proposals: false }
  schedule_allowed: true

2) TASK-QUAL-01
  name: "Detail Depth Opportunities"
  intent: "optimize"
  required_capabilities: ["quality_depth"]
  default_agent_id: "AGT-QUAL-01"
  outputs: { insights: true, bottlenecks: false, recommendations: true, findings: true, proposals: true }
  schedule_allowed: true

3) TASK-QUAL-02
  name: "Template Quality & Determinism Audit"

intent: "audit"
required_capabilities: ["template_audit"]
default_agent_id: "AGT-QUAL-01"
outputs: { insights: true, bottlenecks: false, recommendations: true, findings: true, proposals: true }
schedule_allowed: true

4) TASK-SYS-01
name: "Internal Registry/Drift Monitor"
intent: "monitor"
required_capabilities: ["registry_integrity","drift_scan"]
default_agent_id: "AGT-SYS-01"
outputs: { insights: true, bottlenecks: false, recommendations: true, findings: true, proposals: true }
schedule_allowed: true

5) TASK-COST-01
name: "Token/Compute Waste Hotspots"
intent: "optimize"
required_capabilities: ["monitor","bottleneck_scan"]
default_agent_id: "AGT-OPS-01"
outputs: { insights: true, bottlenecks: true, recommendations: true, findings: false, proposals: false }
schedule_allowed: true

Note: For v1, proposals are optional. If you cannot safely generate diffs, emit recommendations instead.

PART B — CORE ENGINE (engine.ts)
---
1) Add loader helpers in engine.ts:
- loadAgentRegistry(root): returns registry
- loadTaskRegistry(root): returns registry
- listAgents(root): returns active agents
- listTasks(root): returns active tasks

2) Extend run model WITHOUT breaking existing MM runs:
- Add run_kind: "mode" | "task" (default "mode" for existing runs)
- For task runs include:
  - task_ids: string[]
  - agent_id?: string
  - task_outputs?: { insights?: boolean; bottlenecks?: boolean; recommendations?: boolean; proposals?: boolean; findings?: boolean }

3) Add new entrypoints:
- createTaskRun(root, store, input) -> returns run record with run_kind="task"
- executeTaskRun(root, store, run) -> executes tasks and writes outputs

4) Output files for TaskRuns (persist alongside existing run outputs):
In the run folder (same pattern as existing runs):
- insights.json (array)
- bottlenecks.json (object)
- recommendations.json (array)
Optionally:
- proposal_packs.json (reuse existing schema if you generate proposals)
Always:
- proof_bundle.json (include what tasks ran, pinned registries/policies, budgets, timestamps)

5) Implement task dispatcher (MVP, heuristic-based is fine):
- TASK-PERF-01 (Bottleneck Finder):
  - Analyze historical runs from store:
    - longest durations
    - most frequent failures / completed_with_limits
    - repeated finding types
  - Emit:
    - bottlenecks.json = { hotspots: [...], by_mode: {...}, budget_hits: {...} }
    - insights.json = ranked observations + evidence refs (run ids)
    - recommendations.json = actionable next steps (e.g., “tighten budget”, “add cache”, “split detector pack”)
- TASK-QUAL-01 (Detail Depth Opportunities):
  - Scan registries where available:
    - templates registry entries with missing/short description/required sections
    - KL items missing citations or too-short summaries
  - Emit insights + recommendations.
  - If confident and bounded, convert to proposal packs using existing patch types (PT-TPL-01, PT-KL-01, PT-KL-02). If not, keep as recommendations only.
- TASK-SYS-01 can reuse existing MM-01/MM-04 logic:
  - Execute DP-REG-01 + DP-DRIFT-01 internally and summarize as insights (do NOT auto-run those modes; just call the internal functions).
- TASK-COST-01:
  - Use existing run metadata (budgets used, limit hits, repeated runs) as a proxy for waste.
  - Emit bottlenecks + recommendations.

6) Budget enforcement:
- Apply mus.policy.json budgets/caps similarly to MM runs.
- If caps exceeded: status "COMPLETED_WITH_LIMITS" and record limits hit in proof bundle.

7) Ledger:
- Append audit entries for:
  - task run created
  - task run started/completed
  - outputs written
No apply/publish logic is introduced here.

PART C — BACKEND ROUTES (mus-routes.ts)

Add new endpoints (do not remove old ones):

Agents/Tasks:
- GET /api/mus/agents
  -> returns listAgents(MUS_ROOT)
- GET /api/mus/tasks
  -> returns listTasks(MUS_ROOT)

Task runs:
- POST /api/mus/task-runs
  body: { task_ids: string[], agent_id?: string, trigger: "manual"|"scheduled", scope_selectors?: any, budgets?: any }
  rules:
    - trigger "scheduled" allowed ONLY if a schedule entry is enabled that permits those tasks (for v1 you can restrict to manual only; but if you support scheduled, enforce proposal-only and publish-disabled).
  -> calls createTaskRun(...) and persists run record

- POST /api/mus/task-runs/:runId/start
  -> calls executeTaskRun(...)

- GET /api/mus/task-runs
  -> list task runs only (run_kind="task")

- GET /api/mus/task-runs/:runId/outputs
  -> returns { run, findings?, insights?, bottlenecks?, recommendations?, proposals?, proof_bundle }

Safety blocks (must enforce at routes layer too):
- Any attempt to publish from a scheduled-triggered run must be blocked (even if future endpoints exist).
- Automation actor cannot apply/publish (if your auth system tags actors; otherwise stub actor="human" and keep server-side guard ready).

DO NOT CREATE CONTINUOUS RUNNERS.

PART D — DEFINITION OF DONE
---
1) Existing MM endpoints still work (MM-01/MM-04).
2) New endpoints:
  - GET /api/mus/agents returns seeded agents
  - GET /api/mus/tasks returns seeded tasks
  - POST /api/mus/task-runs creates run_kind="task"
  - POST /api/mus/task-runs/:id/start produces:
    - insights.json
    - bottlenecks.json (for perf/cost tasks)
    - recommendations.json
    - proof_bundle.json
3) Task outputs are readable and actionable:
  - Bottleneck Finder identifies hotspots from historical runs (durations, failures, limit hits)
  - Detail Depth task identifies concrete opportunities in templates/KL based on registry content
4) No apply/publish is executed automatically.
5) Clear errors if task_id/agent_id invalid or missing registries.

IMPLEMENTATION NOTE
If your store.ts currently only understands MM runs, extend it minimally:
- store must persist arbitrary run_kind and new output filenames.
Do not refactor entire storage architecture.

DELIVER PATCH
Return the updated mus-routes.ts and engine.ts changes, plus the two new registry JSON files.